

Rapport technique — SDB (Standalone Docker Backup)

Projet : outil autonome de sauvegarde et de restauration des volumes Docker

Version : 0.2 — **Date** : juillet 2026

Stack : Go 1.25 · Vue 3 / TypeScript · restic · SQLite · Docker

Sommaire

1. [Introduction](#)
 2. [Présentation de la solution](#)
 3. [Architecture technique](#)
 4. [Choix technologiques](#)
 5. [Gestion et stockage des données](#)
 6. [Sécurité](#)
 7. [API, temps réel et supervision](#)
 8. [Qualité et industrialisation](#)
 9. [Bilan et perspectives](#)
-

1. Introduction

1.1 Contexte

Dans une infrastructure conteneurisée, l'intégralité de l'état persistant d'une application — bases de données, fichiers déposés par les utilisateurs, configurations — réside dans les **volumes Docker**. Or Docker ne fournit aucun mécanisme natif pour les sauvegarder : en cas de mauvais déploiement, d'erreur humaine ou d'attaque, un volume écrasé est définitivement perdu.

En pratique, les équipes comblent ce manque avec des scripts artisanaux (tar + cron) ou des outils en ligne de commande : pas d'historique centralisé, pas de chiffrement systématique, et une restauration improvisée au pire moment.

1.2 Objectifs du projet

SDB (*Standalone Docker Backup*) répond à ce besoin avec quatre objectifs :

Objectif	Traduction concrète
Autonomie	un seul binaire, interface web embarquée, aucune dépendance à installer
Sécurité	chiffrement de bout en bout, conteneur durci, secrets jamais en clair
SimPLICITÉ	sauvegarde et restauration en un clic, planification intégrée
Observabilité	progression en temps réel, historique complet, métriques Prometheus

1.3 Périmètre

Le projet couvre : la découverte des conteneurs et de leurs volumes, la sauvegarde manuelle ou planifiée vers sept types de destinations, la restauration ciblée d'un volume, la gestion des utilisateurs et des politiques de rétention. Il se limite volontairement à **un hôte Docker** par instance.

2. Présentation de la solution

2.1 Fonctionnalités

- **Sauvegarde à chaud ou à froid** : possibilité d'arrêter le conteneur pendant le snapshot pour une cohérence parfaite, avec redémarrage automatique garanti.
- **Hooks pré/post-sauvegarde** : exécution d'une commande dans le conteneur cible (ex. `pg_dumpall` avant le snapshot d'une base PostgreSQL).
- **Snapshots incrémentaux, dédoublés et chiffrés** (moteur restic).
- **Restauration en un clic** vers le volume d'origine, depuis n'importe quel snapshot, avec arrêt/relance automatique du conteneur consommateur.
- **Planification** : expressions cron 5 champs, activation/désactivation à chaud, déclenchement manuel, suivi de la dernière exécution.
- **Rétention automatique** : politique keep-last / daily / weekly / monthly appliquée après chaque sauvegarde réussie (`restic forget --prune`).
- **Vérification d'intégrité** des dépôts, planifiée (hebdomadaire par défaut) ou à la demande.
- **Multi-utilisateurs** avec deux rôles (administrateur / utilisateur).

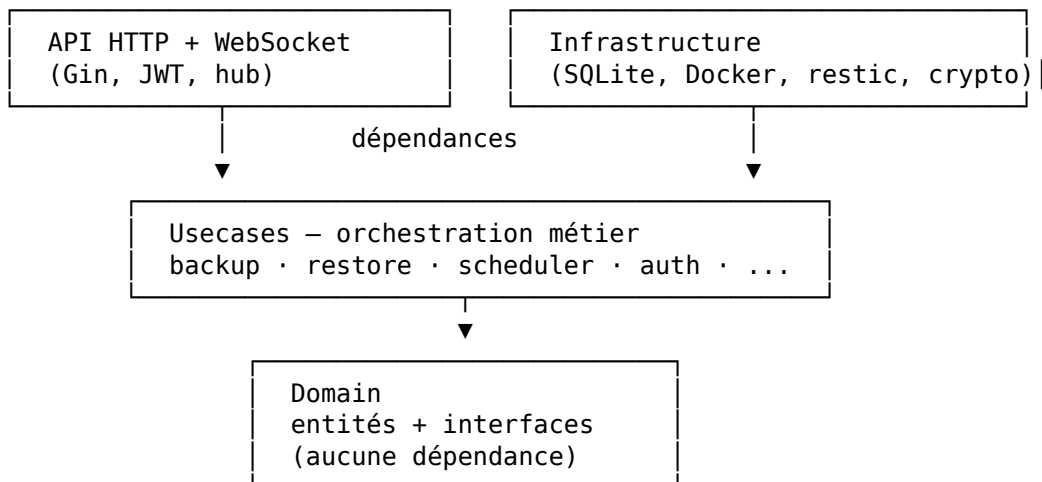
2.2 Interface

Le tableau de bord web (thème sombre) affiche l'état global du système via un indicateur unique (« North Star » : vert / ambre / rouge), la liste des conteneurs avec leurs actions, les opérations en cours avec barre de progression en direct, l'historique filtrable des sauvegardes et restaurations, et la gestion des stockages, planifications et utilisateurs. Les formulaires appliquent le principe de *divulcation progressive* : deux champs essentiels visibles, les options avancées repliées.

3. Architecture technique

3.1 Clean Architecture

Le backend suit les principes de la Clean Architecture : **toutes les dépendances pointent vers le domaine**, jamais l'inverse.



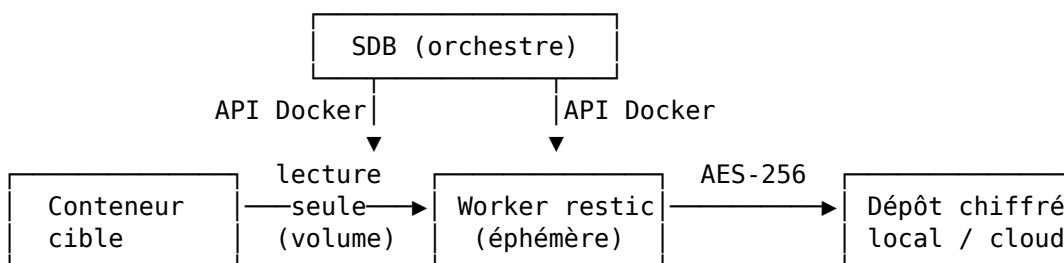
Couche	Répertoire	Rôle
Domain	internal/domain/	entités métier et <i>ports</i> (interfaces)
Usecases	internal/usecase/	règles métier, orchestration des ports
Infrastructure	internal/infra/	adaptateurs SQLite, Docker, restic, crypto
Livraison	internal/api/http/	API REST, WebSocket, authentification

Couche	Répertoire	Rôle
--------	------------	------

Le point d'entrée `cmd/sdb/main.go` assemble les couches au démarrage (injection de dépendances manuelle). Ce découpage rend le cœur métier **testable sans Docker ni base de données** : les 63 tests automatisés du projet s'exécutent sur des implémentations factices des ports.

3.2 Le worker éphémère

Le choix d'architecture central : **SDB ne lit jamais les données des volumes**. Pour chaque opération, il crée via l'API Docker un conteneur jetable (image officielle `restic/restic`) qui exécute le travail puis est systématiquement détruit.



Propriétés obtenues :

- les volumes sont montés **en lecture seule** dans le worker (sauvegarde) ;
- les secrets du dépôt (mot de passe `restic`, clés `cloud`) sont injectés en mémoire dans le worker et disparaissent avec lui ;
- pour un dépôt local, le worker tourne **sans aucun accès réseau** ;
- les fichiers sensibles (clé SSH, compte de service Google) sont copiés dans le worker par l'API Docker en permissions `0600`, sans jamais toucher le disque de l'hôte.

3.3 Pipeline d'une sauvegarde

L'API répond `202 Accepted` immédiatement ; le pipeline s'exécute dans une goroutine dédiée, annulable, dont la progression est diffusée en `WebSocket` :

1. **Pre-hook** dans le conteneur cible (politique par défaut : un échec annule la sauvegarde — mieux vaut pas de snapshot qu'un snapshot incohérent) ;
2. **Arrêt optionnel** du conteneur (sauvegarde à froid) ;
3. **Snapshot** via le worker éphémère ;
4. **Redémarrage garanti** : exécuté sur un contexte insensible à l'annulation — quoi qu'il arrive (échec, crash, annulation), le conteneur de production repart ;
5. **Post-hook** (nettoyage) — un échec produit un avertissement, pas un échec ;
6. **Rétention** (`forget --prune`) après succès uniquement.

Garanties complémentaires : une seule sauvegarde à la fois par conteneur, une seule restauration à la fois par volume, et au redémarrage de SDB les opérations interrompues par un arrêt brutal sont automatiquement marquées en échec.

3.4 Frontend

Le frontend `Vue 3` (`Composition API`, `TypeScript strict`) est **compilé puis embarqué dans le binaire Go** (`go:embed`) : en production, un seul exécutable sert l'API et l'interface. L'état global est géré par `Pinia` (session, santé, flux d'événements) ; le fichier `types.ts` est le miroir exact des DTO Go, ce qui fige le contrat API côté client.

4. Choix technologiques

Choix	Alternative écartée	Justification
Go	Node.js, Python	binaire statique unique, concurrence native (goroutines)
restic	moteur maison, tar	chiffrement, déduplication et incrémental éprouvés
SQLite (modernc, pur Go)	PostgreSQL, driver CGO	zéro dépendance externe, compilation statique sans CGO
Vue 3 + TypeScript	React	Composition API concise, typage strict vérifié en C
Gin	net/http seul	routage, binding JSON et middlewares matures
Image distroless	Alpine, Debian	pas de shell ni de gestionnaire de paquets dans l'image

Dépendances verrouillées : `go.sum` et `package-lock.json` sont versionnés, les builds sont reproductibles (`go mod download`, `npm ci`).

5. Gestion et stockage des données

5.1 Trois niveaux de persistance

Donnée	Emplacement
Métadonnées (comptes, historiques, planifications, configs de stockage)	SQLite, volume Docker <code>/data</code>
Sauvegardes (le contenu des volumes)	dépôts restic : disque local, S3, EBS
Secrets d'exploitation (clé maître, secret JWT)	fichiers montés (Docker secrets)

5.2 Modèle de données

Cinq tables principales, avec migrations SQL versionnées et embarquées dans le binaire (appliquées automatiquement au démarrage, transactionnelles, avec contrôle d'intégrité référentielle) :

- `users` — comptes (hash Argon2id, rôle);
- `storage_configs` — destinations (identifiants et mot de passe restic en blobs chiffrés);
- `backups_history` / `restores_history` — journal de chaque opération (statut, octets, snapshot, horodatage, journal d'erreur);
- `backup_schedules` — planifications (cron, options en JSON, dernière exécution).

5.3 Point critique : la clé maître

La clé maître (`SDB_MASTER_KEY`) chiffre les identifiants de stockage au repos. Sa perte rend ces configurations indéchiffrables — elle doit être sauvegardée séparément. Les sauvegardes elles-mêmes restent restaurables directement avec restic tant que le mot de passe du dépôt est connu.

6. Sécurité

La sécurité était la priorité n° 1 du cahier des charges. Mesures par couche :

Authentification et comptes

- mots de passe hachés **Argon2id** (paramètres OWASP), comparaison en temps constant, protection contre l'énumération de comptes au login;
- jetons **JWT HS256 avec algorithme épinglé** (bloque l'attaque par confusion d'algorithme), *rate-limiting* du login (10 tentatives/min/IP);
- invariant métier : impossible de supprimer ou rétrograder le dernier administrateur.

Secrets

- chiffrement **AES-256-GCM** (authentifié) de tous les secrets au repos, nonce aléatoire par opération, octet de version pour l'agilité future ;
- mot de passe de dépôt restic **généré automatiquement** (43 caractères) et **immuable** (le changer verrouillerait le dépôt).

Conteneur et réseau

- conteneur SDB durci : rootfs **read-only**, **cap_drop ALL**, **no-new-privileges**, /tmp en tmpfs, image distroless ;
- port publié sur **127.0.0.1 uniquement** — la publication de port Docker contourne UFW/iptables, ce piège est neutralisé par construction ;
- connexion tcp:// au démon Docker **refusée au démarrage** sans mTLS complet (CA + certificat + clé) ;
- WebSocket : vérification d'origine *same-origin* stricte (anti-hijacking).

Le risque assumé : SDB monte le socket Docker, ce qui équivaut à un accès root sur l'hôte. C'est inhérent à sa fonction d'orchestrateur ; le durcissement ci-dessus vise à ce que SDB lui-même ne soit pas un point d'entrée.

7. API, temps réel et supervision

7.1 API REST

31 endpoints sous /api/v1, authentifiés par JWT (sauf le login), avec autorisation par rôle sur les mutations sensibles. Les opérations longues répondent 202 Accepted et se suivent en temps réel. Les erreurs métier se projettent systématiquement sur les codes HTTP (400, 401, 403, 404, 409, 503) ; les erreurs inattendues renvoient un 500 opaque, le détail restant dans les journaux.

Famille	Exemples
Authentification	POST /auth/login
Conteneurs	GET /containers, GET /health
Stockages	CRUD + GET /storage/:id/snapshots, POST /storage/:id/check
Sauvegardes	POST /backups → 202, DELETE /backups/:id, historique
Restaurations	POST /restores → 202, annulation, historique
Planifications	CRUD + POST /schedules/:id/run
Utilisateurs	CRUD (admin), changement de mot de passe (soi-même ou admin)

7.2 Temps réel : le hub WebSocket

Un *hub* central diffuse les événements de progression à tous les navigateurs connectés. Sa propriété essentielle : **il ne bloque jamais les sauvegardes**. Une seule goroutine possède la liste des clients (aucun verrou) ; un client trop lent est déconnecté plutôt qu'attendu ; un hub saturé abandonne l'événement (l'état faisant foi est en base). Côté client, la reconnexion est automatique avec *backoff* exponentiel (1 s → 30 s, avec aléa).

7.3 Supervision : Prometheus

L'endpoint /metrics (désactivé par défaut, protégé par un jeton statique comparé en temps constant) expose notamment `sdb_backups_total{status}`, `sdb_restores_total{status}`, `sdb_running_jobs` et `sdb_last_backup_success_timestamp_seconds{container}` — cette dernière permet l'alerte la plus utile : « aucune sauvegarde réussie depuis 24 h pour ce conteneur ». Le collecteur consomme le même flux d'événements que le hub : son ajout n'a modifié aucune ligne de logique métier.

8. Qualité et industrialisation

8.1 Tests

63 tests automatisés (~2 000 lignes), exécutés avec le détecteur de concurrence de Go. Grâce à l'architecture en ports, le pipeline complet de sauvegarde est testé **sans démon Docker** : rollback après échec, politique des hooks, conflits d'exécution simultanée, annulation, échec de redémarrage, non-fuite des secrets (vérification qu'aucun secret en clair n'apparaît dans le fichier SQLite), sécurité JWT (rejet d'un jeton alg=none), éviction des clients WebSocket lents.

8.2 Validation en conditions réelles

Le cycle complet a été validé de bout en bout contre un démon Docker réel : planification cron déclenchée à la seconde près, politique de rétention vérifiée (15 sauvegardes ramenées à 3 snapshots), et scénario complet *sauvegarde* → *destruction des données* → *restauration à l'identique* (contenus et horodatages vérifiés).

8.3 Intégration et livraison continues

- **CI GitHub Actions** en trois jobs : backend (go vet, tests avec race detector, build), frontend (vue-tsc strict, build Vite), et construction de l'image Docker complète.
- **Image Docker multi-stage** : build du frontend (Node) → compilation Go statique avec le frontend embarqué → image finale distroless.
- **Déploiement en une commande** : docker compose up -d --build, secrets générés au préalable, migrations et compte administrateur initial créés automatiquement au premier démarrage.

8.4 Chiffres clés

Indicateur	Valeur
Lignes de code (Go + TypeScript/Vue, hors tests)	≈ 8 800
Lignes de tests	≈ 2 000
Tests automatisés	63
Endpoints API	31
Destinations de stockage	7
Taille de l'image finale	≈ 35 Mo

9. Bilan et perspectives

9.1 Bilan

Le projet livre un outil **fonctionnel de bout en bout**, validé en conditions réelles, qui tient ses quatre objectifs : autonome (un binaire, une commande de déploiement), sécurisé (chiffrement systématique, conteneur durci, défense en profondeur), simple (sauvegarde et restauration en un clic) et observable (temps réel, historique, métriques).

Au-delà des fonctionnalités, le principal acquis est architectural : la Clean Architecture et le pattern du worker éphémère donnent un système où le cœur métier est testé isolément, où chaque brique technique est remplaçable, et où l'outil de sauvegarde n'a jamais accès en écriture aux données qu'il protège.

9.2 Limites actuelles

- un seul hôte Docker par instance ;
- pas de notification proactive en cas d'échec (l'information est dans l'historique et les métriques, mais rien n'est « poussé ») ;
- TLS à assurer par un reverse proxy pour un accès distant.

9.3 Perspectives

1. **Notifications** d'échec par e-mail et webhook ;
2. **Multi-hôtes** : piloter plusieurs démons Docker (mTLS déjà supporté) ;
3. **TLS natif** sur l'interface ;
4. **Publication open source** du projet.

Document rédigé dans le cadre de la soutenance du projet SDB — juillet 2026.